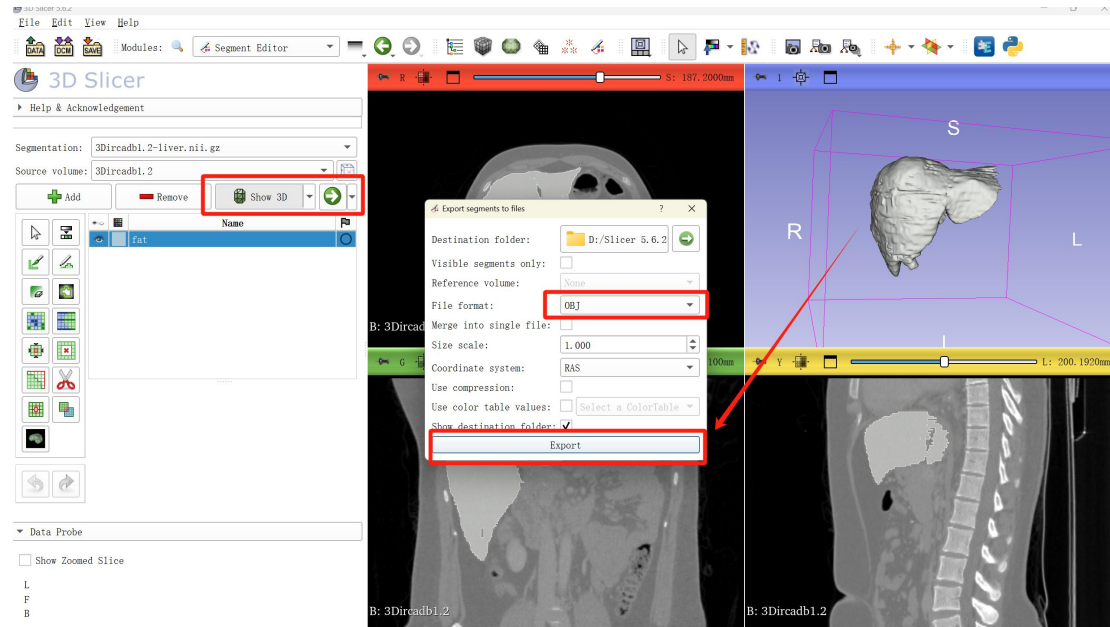


1. Generate .obj Mesh Data from .nii.gz Files

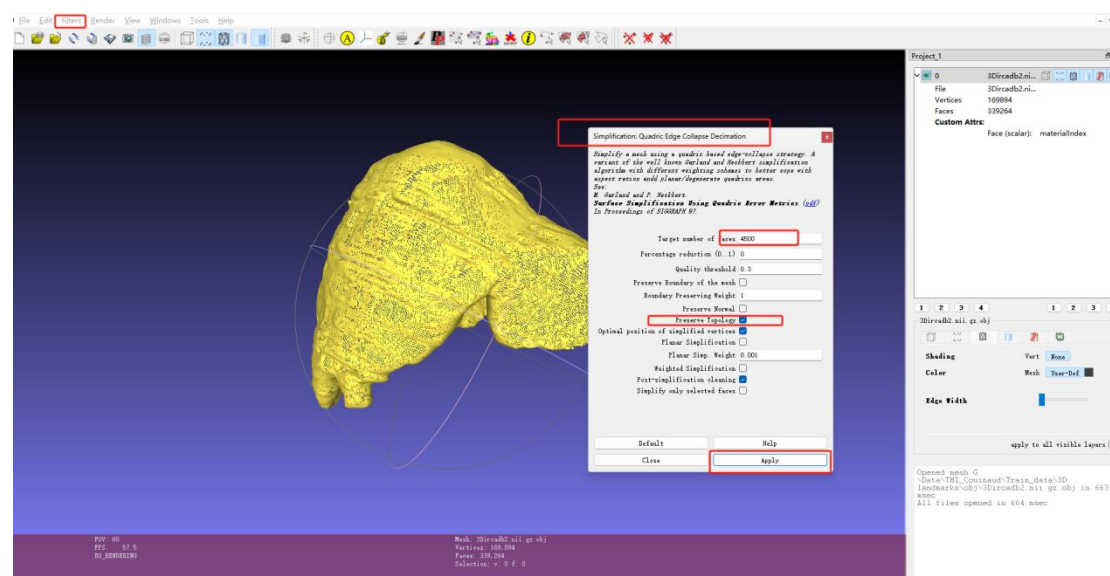
Using **3D Slicer** software, import the liver Image and corresponding Mask. Then, export the 3D Mask of the liver as Mesh data in .obj format, as shown in the figure below.



2. Preprocess the Mesh Data in MeshLab

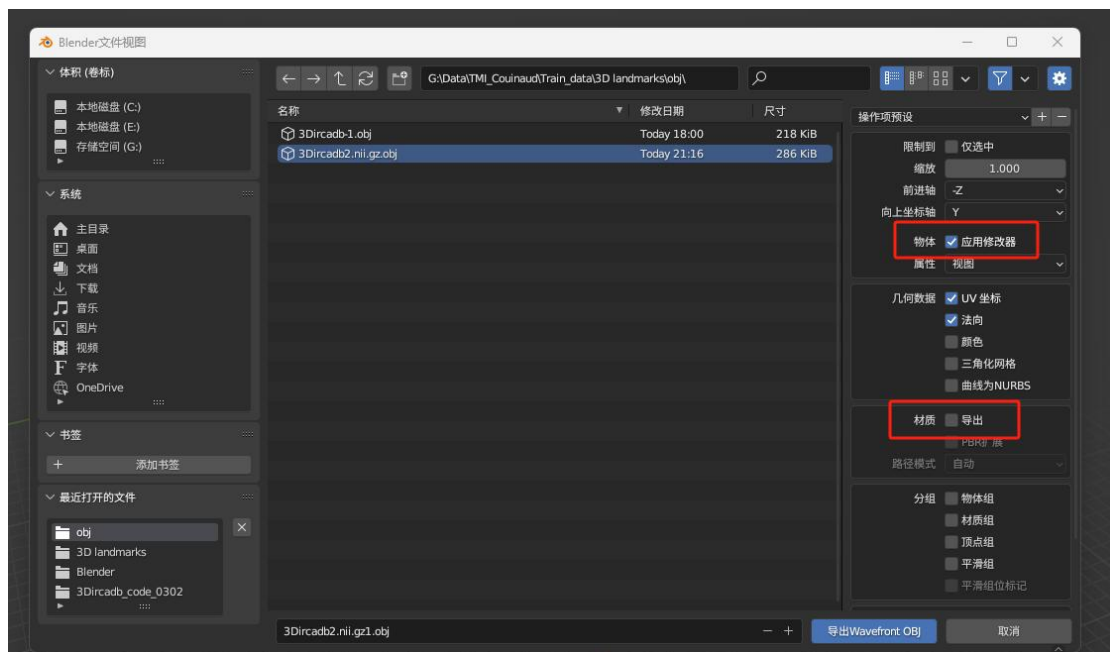
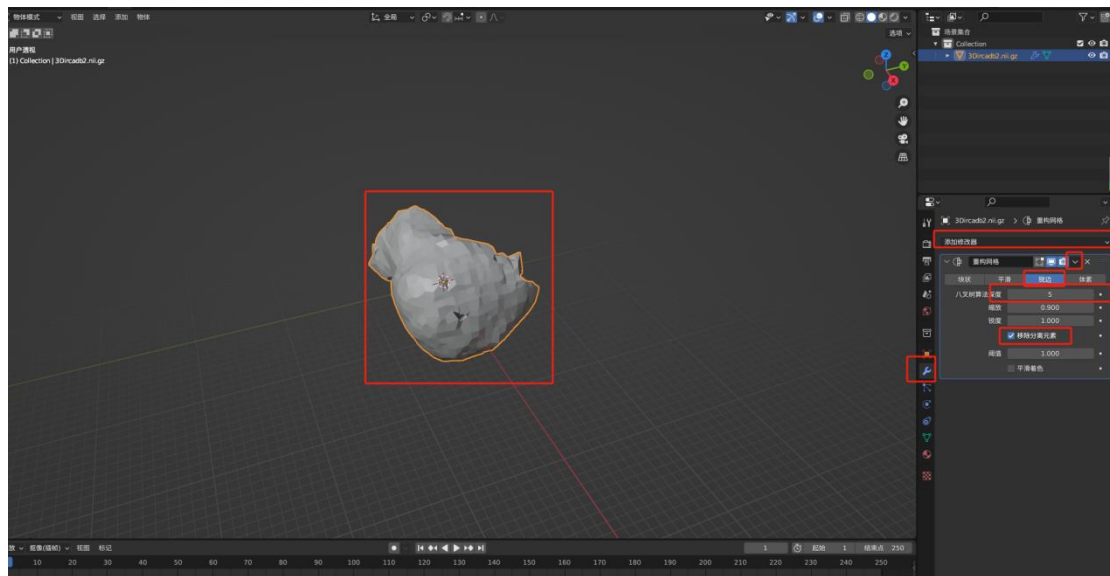
Use **MeshLab** software to preprocess the Mesh by performing the following tasks:

- **Face Flipping**
- **Mesh Simplification** Afterward, export the new Mesh data.



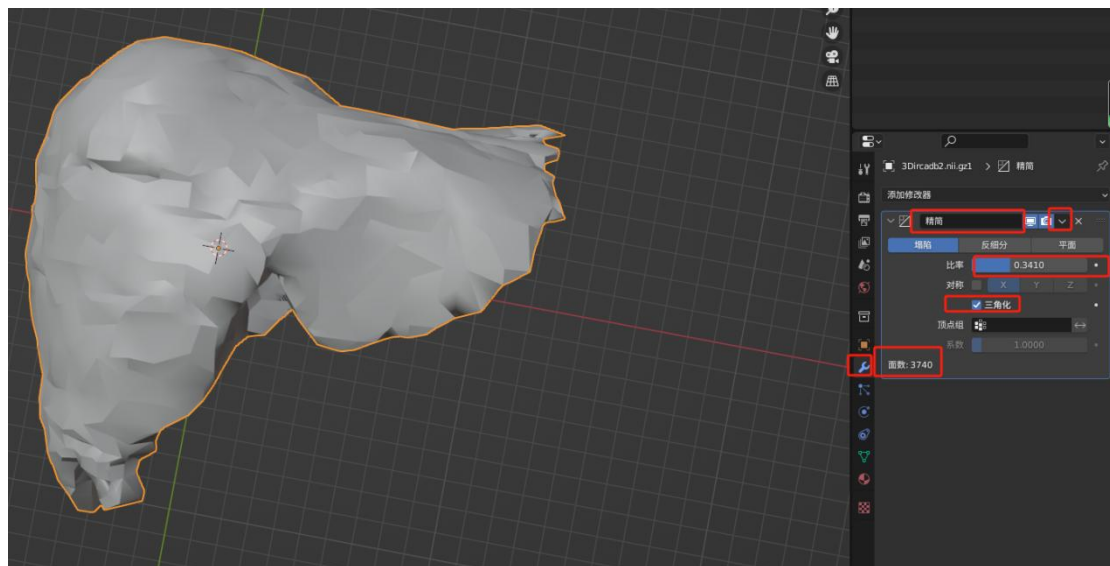
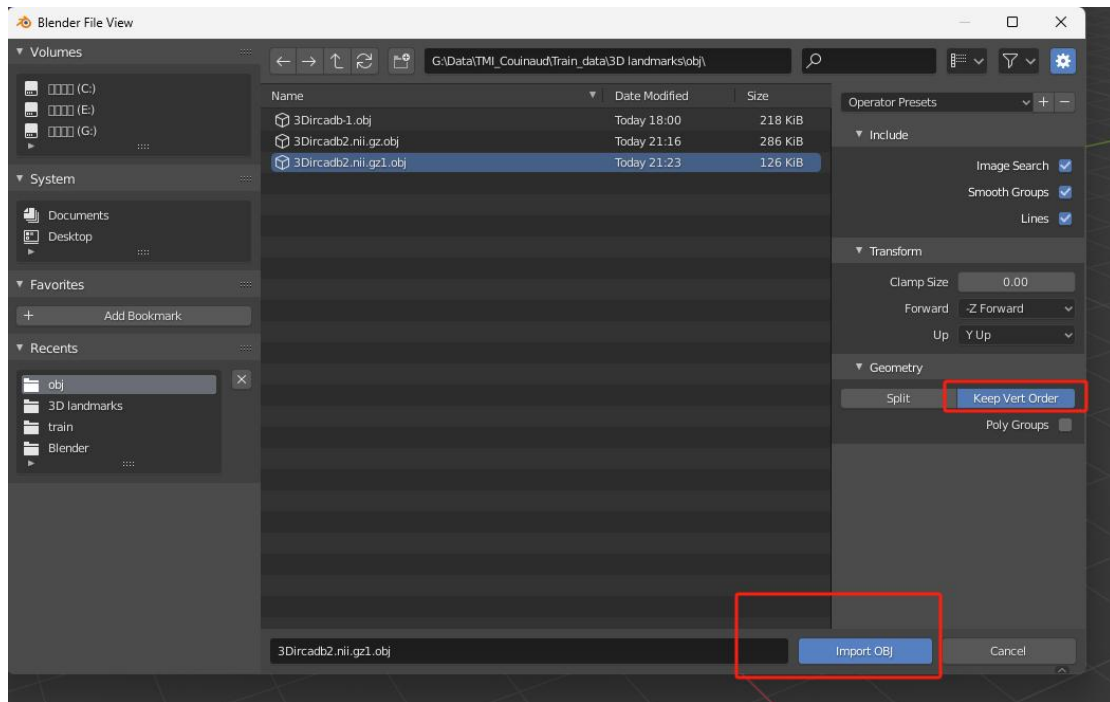
3. Mesh Reconstruction in Blender

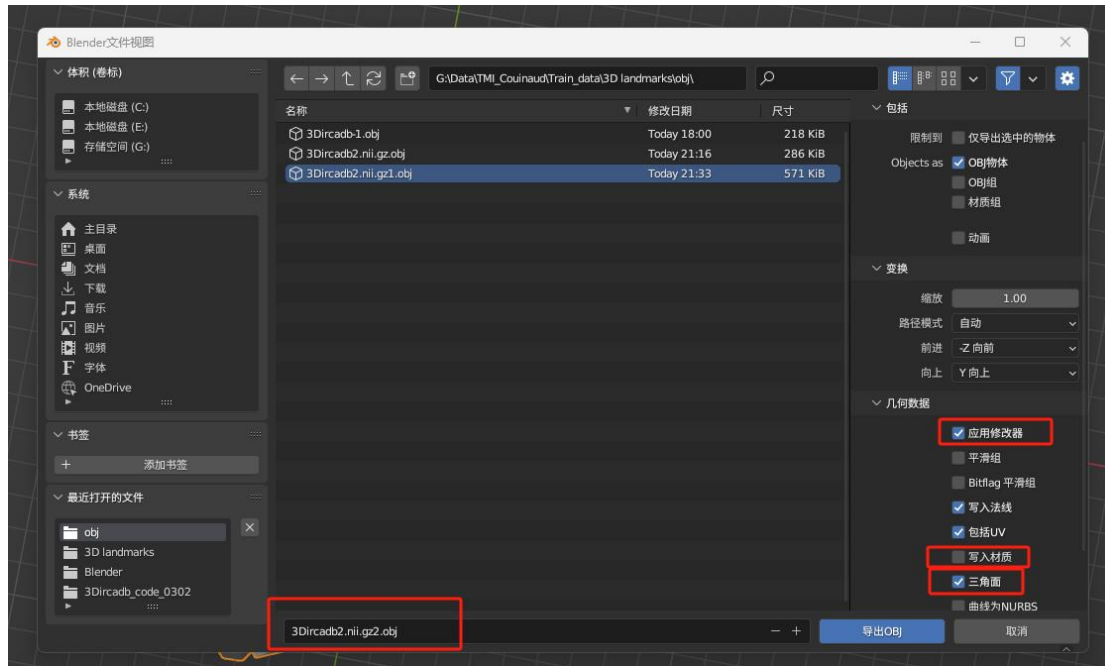
Import the rough Mesh from the previous step into **Blender** software. Use the modifiers to reconstruct the mesh and apply the reconstruction. This will generate a new, improved Mesh, which can be exported (no need to export materials).



4. Simplify the Mesh and Export the Data in Blender

Import the Mesh from the previous step into **Blender**. Select the option to "Preserve Vertex Order". Then, apply the **Simplify Modifier** to convert the mesh into a "triangular mesh" form. Specify the number of triangles, and export the new mesh (materials are not necessary).





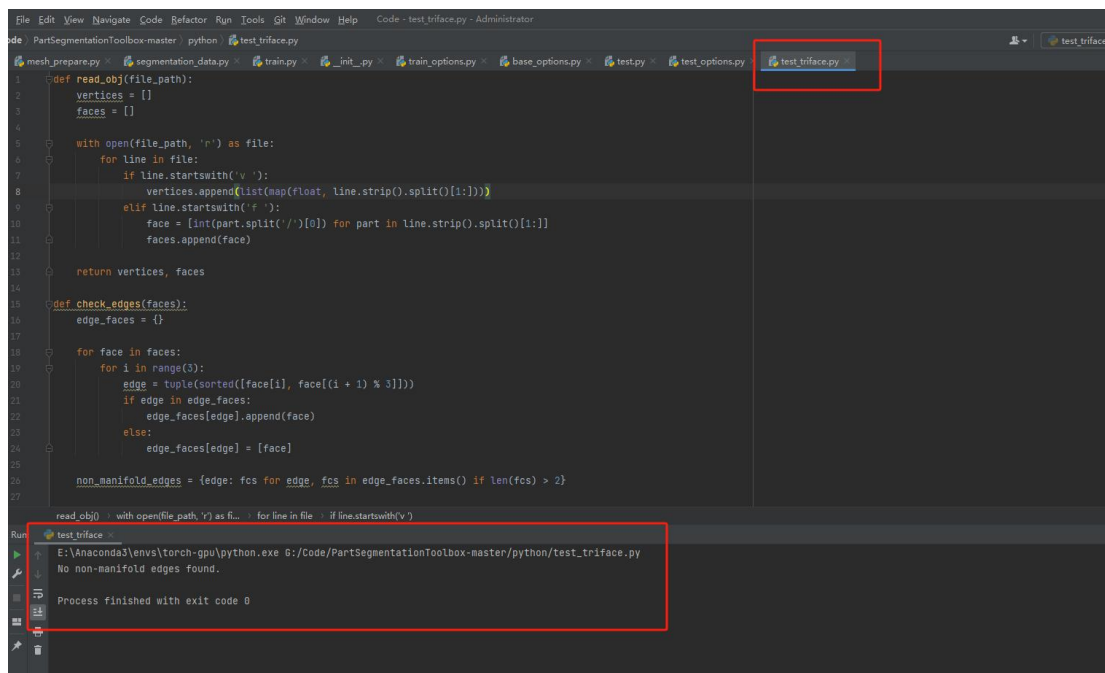
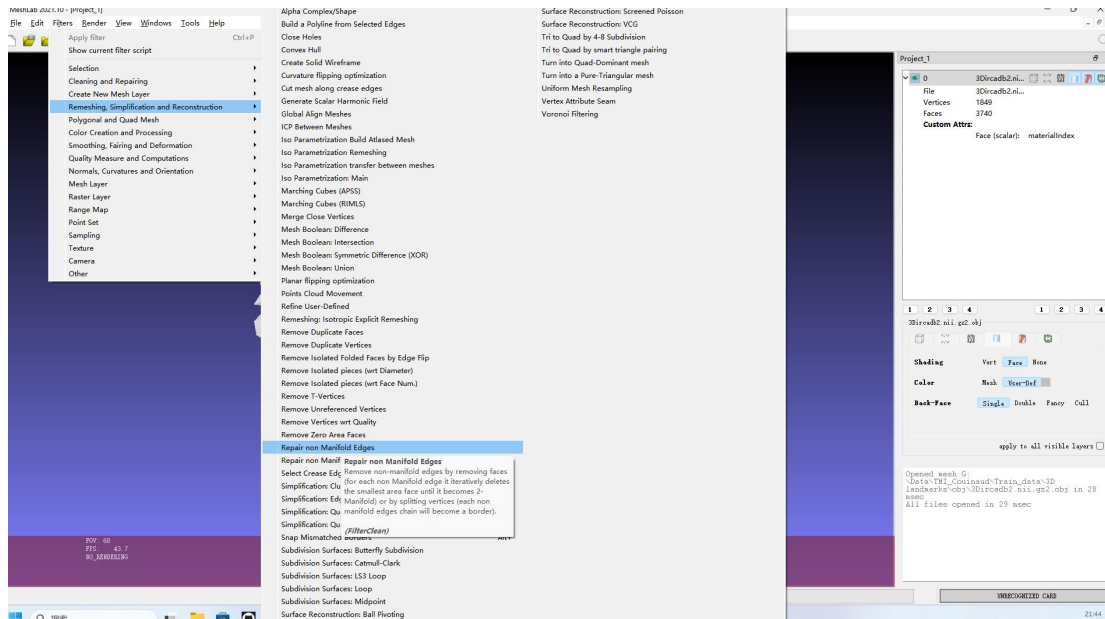
5. Check for Non-Manifold Edges and Repair the Mesh

Check if there are any **non-manifold edges**. If any are found, import the Mesh into **MeshLab** again and use the "Fix Non-Manifold Edges" function. Export the repaired mesh and verify that there are no non-manifold edges.

```

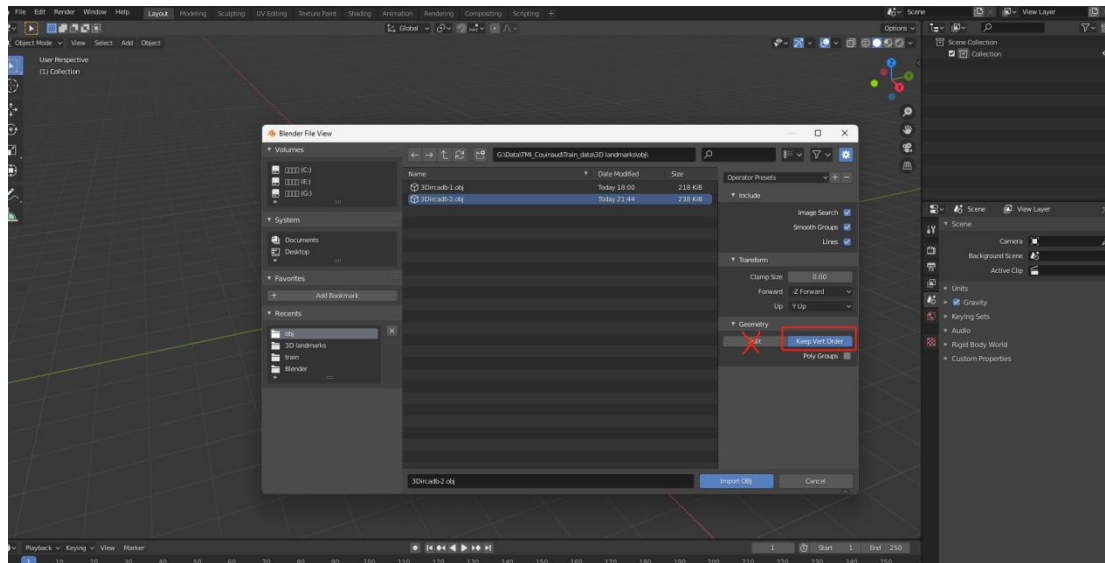
File Edit View Navigate Code Refactor Run Tools Git Window Help Code · test_triface.py · Administrator
Code PartSegmentationToolbox-master python · test_triface.py
mesh_prepare.py segmentation_data.py train.py _init_.py train_options.py base_options.py test.py test_options.py test_triface.py
1 def read_obj(file_path):
2     vertices = []
3     faces = []
4
5     with open(file_path, 'r') as file:
6         for line in file:
7             if line.startswith('v '):
8                 vertices.append(list(map(float, line.strip().split()[1:])))
9             elif line.startswith('f '):
10                face = [int(part.split('/')[0]) for part in line.strip().split()[1:]]
11                faces.append(face)
12
13    return vertices, faces
14
15 def check_edges(faces):
16     edge_faces = {}
17
18     for face in faces:
19         for i in range(3):
20             edge = tuple(sorted([face[i], face[(i + 1) % 3]]))
21             if edge in edge_faces:
22                 edge_faces[edge].append(face)
23             else:
24                 edge_faces[edge] = [face]
25
26     non_manifold_edges = {edge: fcs for edge, fcs in edge_faces.items() if len(fcs) > 2}
27
Run test_triface
Edge (615, 647) belongs to faces [[647, 629, 615], [800, 647, 615], [615, 615, 647], [644, 647, 615]]
Edge (708, 712) belongs to faces [[707, 712, 708], [712, 710, 708], [711, 712, 708], [676, 708, 712]]
Edge (779, 780) belongs to faces [[778, 780, 779], [780, 768, 779], [794, 780, 779], [779, 780, 777]]
Edge (855, 857) belongs to faces [[857, 851, 855], [855, 515, 857], [857, 509, 855], [857, 855, 833]]
Edge (509, 855) belongs to faces [[857, 509, 855], [833, 855, 509], [615, 855, 509], [509, 855, 850]]
Edge (1037, 1044) belongs to faces [[1036, 1044, 1037], [1040, 1044, 1037], [1044, 1041, 1037], [1042, 1037, 1044]]
Edge (1134, 1140) belongs to faces [[1137, 1140, 1134], [1134, 1140, 1138], [1134, 1140, 1146], [1093, 1140, 1134]]
Edge (1366, 1370) belongs to faces [[1366, 1371, 1370], [1370, 40, 1366], [1366, 1370, 1368], [1370, 1366, 1363]]
Edge (1671, 1674) belongs to faces [[1674, 1672, 1671], [1675, 1674, 1671], [1673, 1674, 1671], [1671, 1674, 1540]]
Process finished with exit code 0

```



6. Label the Dataset in Blender

Next, use **Blender** to annotate the dataset (label the edges). Import the processed Mesh (make sure to preserve the vertex order), and begin the labeling process.



7. Create Vertex Groups for Edge Labeling

In **Edit Mode**, run the grouping script to create two vertex groups. These vertex groups will store the indices for:

- **Falciform Ligament (Group 1)**
- **Lower Edge of the Liver (Group 2)**

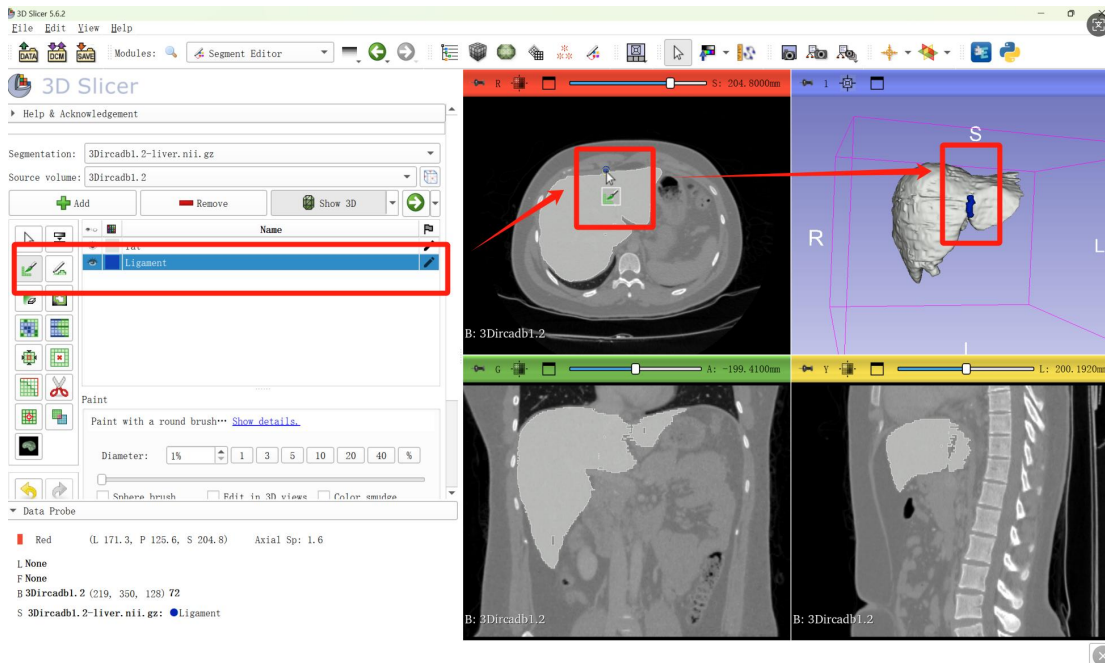
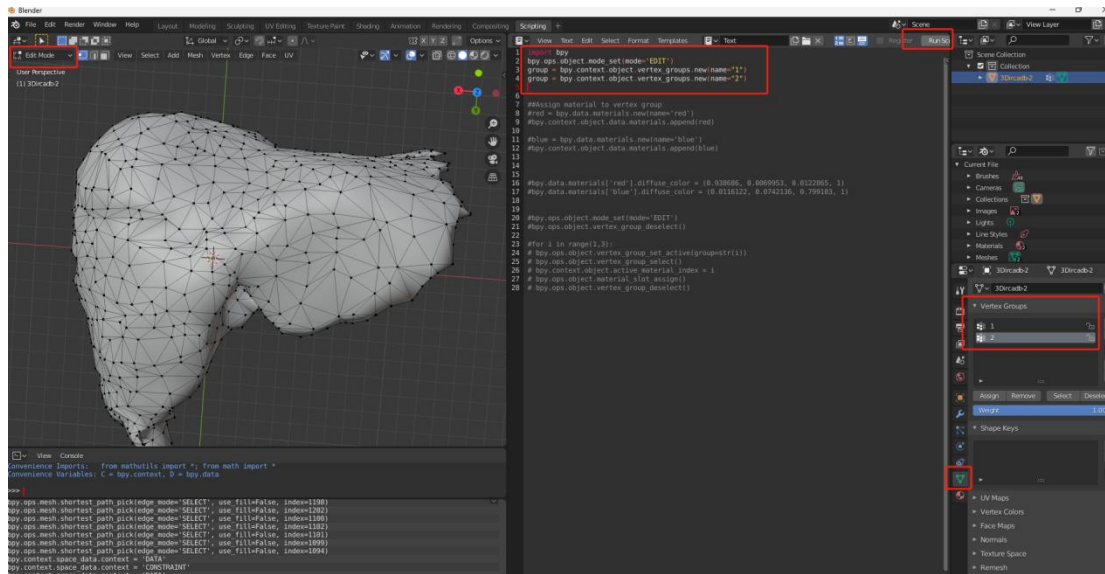
Annotation Method:

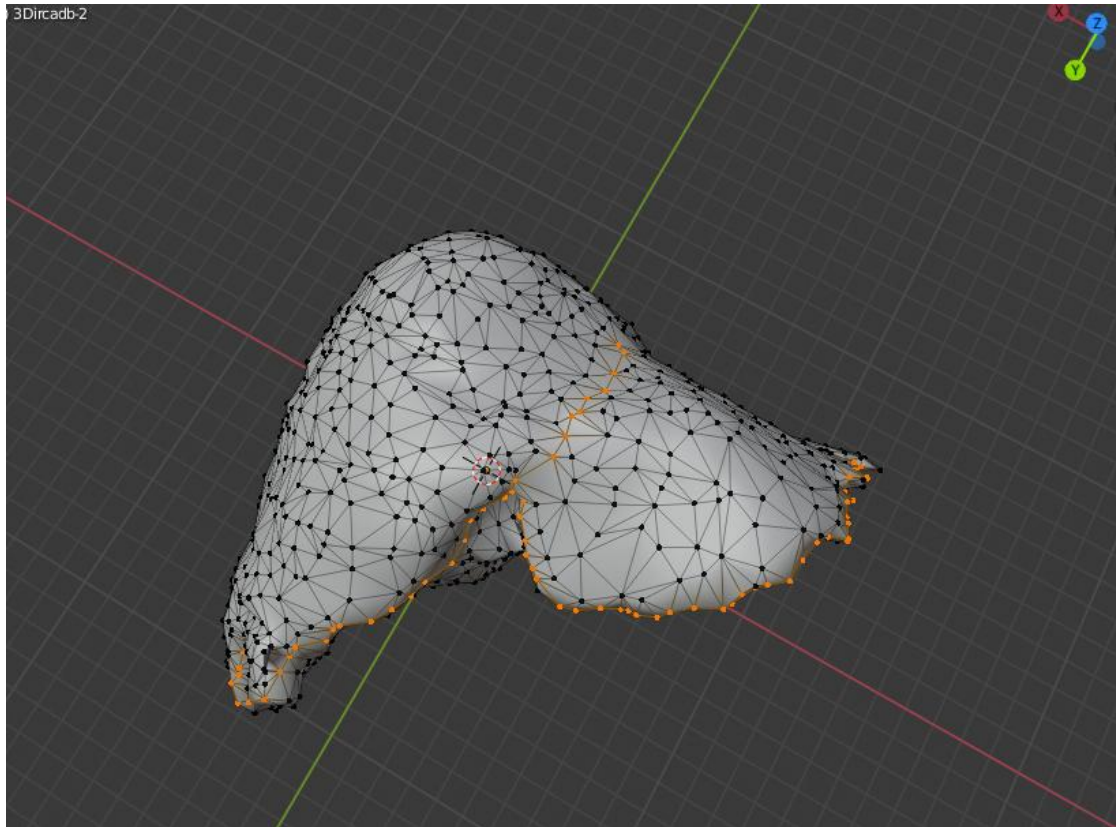
Firstly, open the CT images and liver label in 3D Slicer, and simultaneously display the 3D mesh view of the liver.

Then, adjust the window width and level of the CT images to locate the positions of the falciform ligament and the liver spine in the 2D view.

Subsequently, in the 2D view, use the annotation tool to label the regions of the falciform ligament and the liver spine. Make sure to check the option for 3D synchronization. With this method, the annotated falciform ligament and liver spine will appear in the 3D view (as a reference for annotating vertex positions in Blender).

Finally, strictly based on the landmark positions in the 3D view of 3D Slicer, annotate the landmark vertices on the processed mesh in Blender.



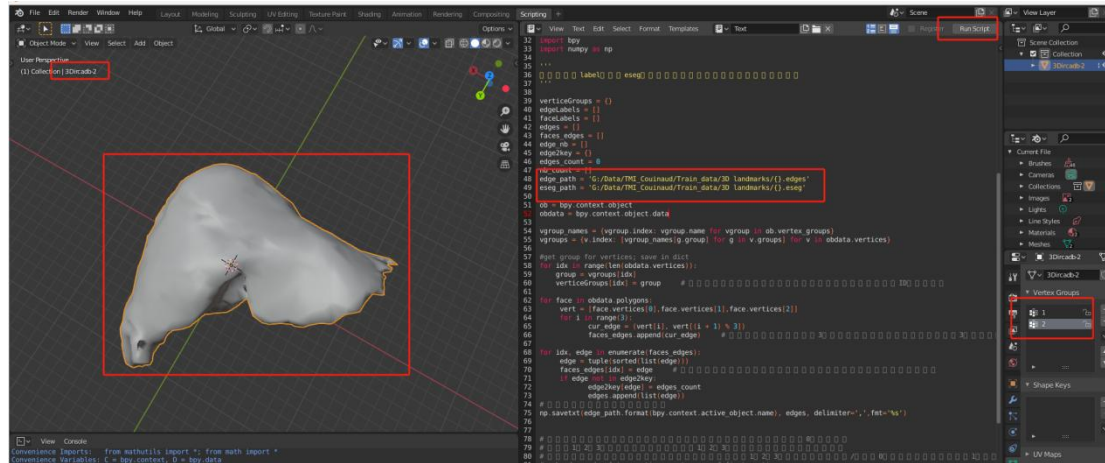


8. Save Edge Labels as .eseg File

In **Object Mode**, run the script to save the two vertex groups as an **.eseg** file, which will serve as the edge label file.

Note:

- (1) The label values should be "0", "1", and "2", where "0" represents the background class.
- (2) It should be noted that while the operations are performed on vertices in Blender, the provided code will convert these into annotations for the edges connected to the vertices, serving as the training labels for the Edge model (indices of the edges and their corresponding categories).

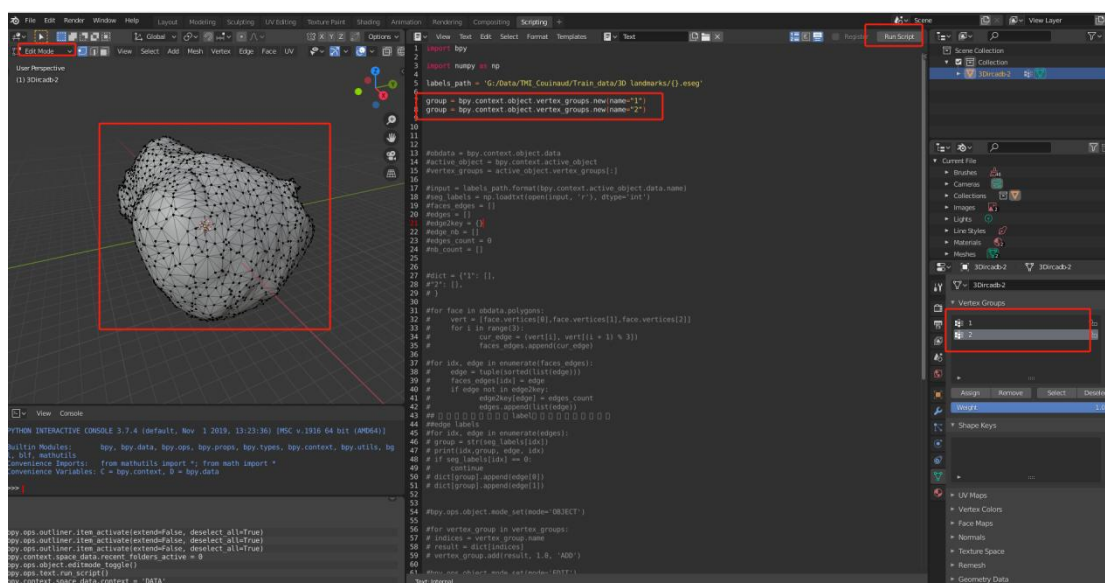


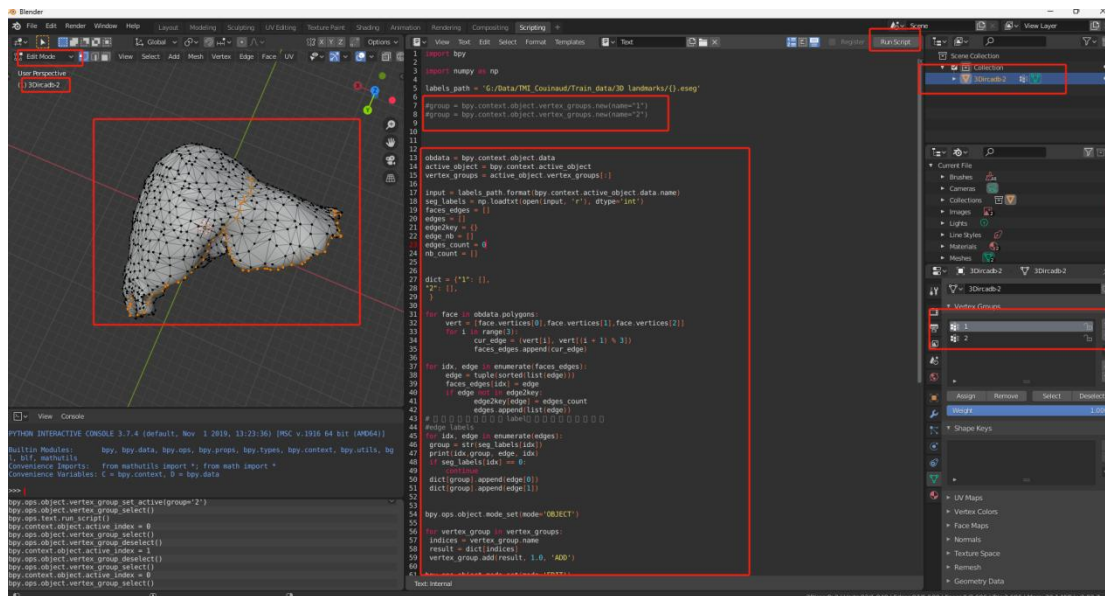
9. Verify Vertex Labeling

To verify the correctness of the vertex labeling, reopen **Blender** and import the Mesh data processed in Step 5, ensuring the vertex order is maintained. Then, in **Edit Mode**, run the following two lines of code to create two empty vertex groups. After commenting out these lines of code, run the subsequent script. This will load the **.eseg** file generated in Step 8 and visualize the vertex/edge labels on the current Mesh model surface. This step confirms that the label positions and order are correct.

At this point, the data labeling process is complete, and you will have obtained:

- The processed **.obj** file
- The edge label file in **.eseg** format
- The **.edges** file containing all edges of the model





Summary:

The overall process starts with extracting 3D mesh data from a segmented mask file. Then, the mesh data is processed to fix various types of errors in the data, followed by mesh simplification for easier annotation and network training. Finally, Blender is used to manually annotate the mesh, and the label files are saved for further use.